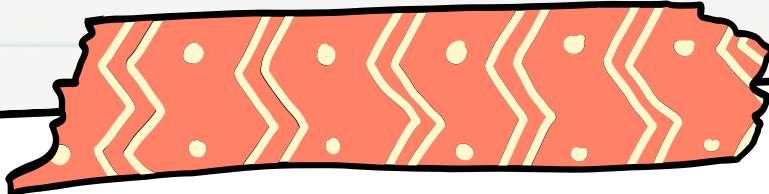




Proposal Finale

Presented by: Group 5 ([makabaka](#))





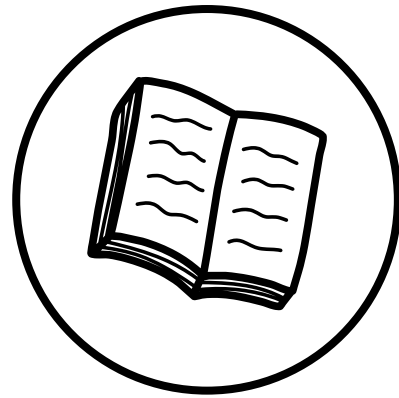
Project Description

Purpose of System

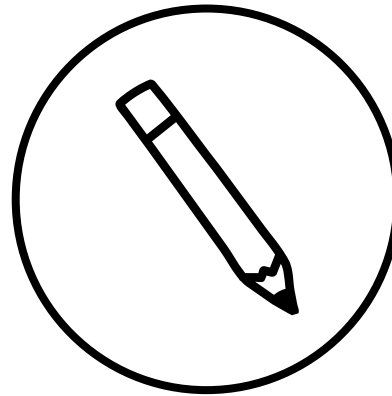
- An effort to **share knowledge** and **promote resource availability** for students who struggle to gain access to professional readings and materials
- Unlike libraries, the reference book exchange system we proposed **allows access to all**.
- The suggested reference book exchange system is to offer a comprehensive platform for users to share, borrow, and lend reference books to transform access to academic resources.
- The purpose of this system is to help users, especially students and
- scholars have the access to educational resources or some limited edition reference books without spending any.



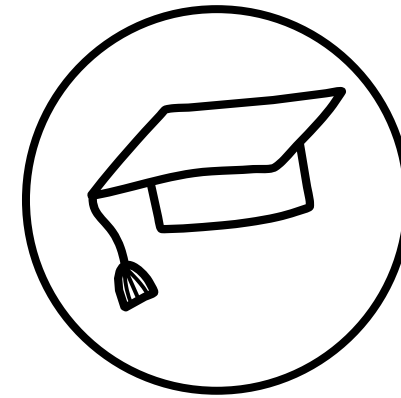
System objectives



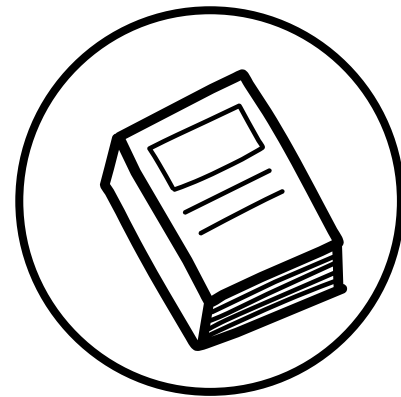
knowledge sharing



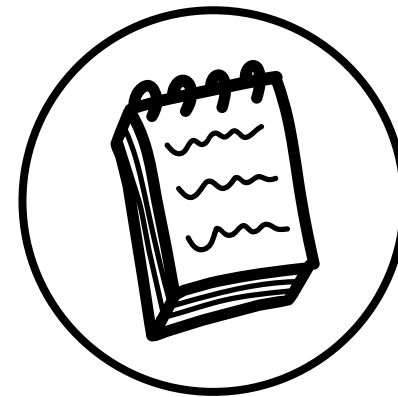
environmental
sustainability



expanding sharing scope



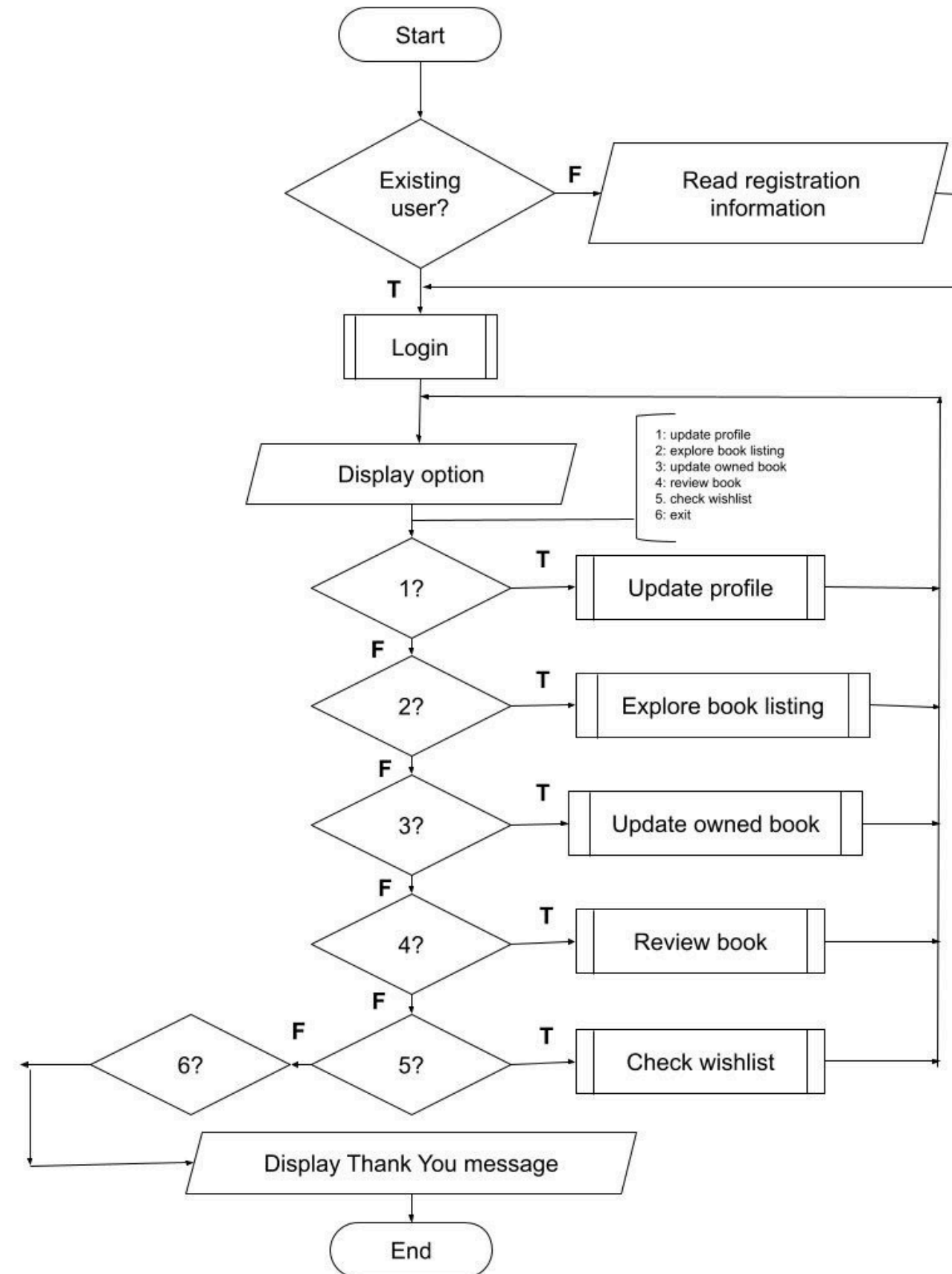
secure platform



facilitate learning



Project Design





Project Specification



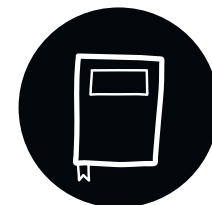
Array of Objects

- allUser object array in main



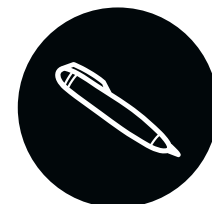
Encapsulation

- every class



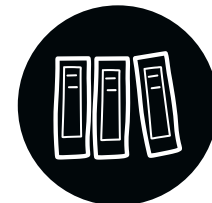
Inheritance

- Book - Dictionary & Encyclopedia
- User - Student & NonStudent



Polymorphism

- dispBookDetails()



Association

- relationship between user and list of books
 - aggregation - User - owned, exchanged, wishlist, review
 - composition - User - personallInfo



Composition

- User - personallInfo



Array of Objects

```
int main() {  
  
    int ucount = 0; //total user count  
    User* allUser[10];  
    for (int i = 0; i < 10; ++i) {  
        |    allUser[i] = new User();  
    }  
  
    int user_i; //current user index  
    int bcount = 0; //total book count
```

- Used in the main class
- To store all information of users
- Limit of user in allUser = 10

Encapsulation

- Used in every class
- combine attributes and methods of that particular class.
- Example: class User used encapsulation to combine all its attributes and methods.

```
263 class User {
264     protected:
265         int userID;
266         string password;
267         PersonalInfo pI;
268         Book **owned;
269         Book **exchanged;
270         Review *re;
271         Wishlist *wish;
272
273     public:
274         int booknum;
275         int wishnum;
276         int exnum;
277
278         User(int ID=-1, string pass="", PersonalInfo _pI = PersonalInfo()){
279             userID = ID;
280             password = pass;
281             pI = _pI;
282             owned = NULL;
283             re = NULL;
284             wish = NULL;
285             booknum = 0;
286             wishnum = 0;
287             exnum = 0;
311         void dispBookList() {
312             for (int i=0; i < booknum; i++){
313                 owned[i]->dispBookDetails();
314             }
315         }
316     }
317
318     void updateBookInfo() {
319         int temp;
320         int select;
321
322         cout << "Enter book ID to update book info: ";
323         cin >> temp;
```

```
class User {  
    protected:  
        int userID;  
        string password;  
        PersonalInfo pI;  
        Book **owned;  
        Book **exchanged;  
        Review *re;  
        Wishlist *wish;
```

Composition

- The `**User**` object contains a `**personalInfo**` object that stores the user's personal details
- If the `**User**` object is destroyed, the `**personalInfo**` is also deleted, indicating a composition relationship

```
class User {  
    protected:  
        int userID;  
        string password;  
        PersonalInfo pI;  
        Book **owned;  
        Book **exchanged;  
        Review *re;  
        Wishlist *wish;
```

Aggregation

- **owned** book array storing owned books
- **exchanged** book array for books exchanged with other users
- **review** object storing user reviews
- **wishlist** array storing book titles for future reference

Inheritance - Book class

- Dictionary and Encyclopedia classes inherit from the Book class
- sharing attributes like title, publisher, publicationYear, ISBN, condition, and status.

```
class Dictionary: public Book {  
    private:  
        string pair_language;  
        string authorTeam;  
  
    public:  
        Dictionary(int ID=0, string tt="", string ps="", int py=0, string con="", string isbn=""  
            pair_language = pl;  
            authorTeam = auT;  
        }  
}
```

```
class Encyclopedia: public Book {  
    private:  
        string author;  
        string language;  
  
    public:  
        Encyclopedia(int ID=0, string tt="", string ps="", int py=0, string con="", string isbn=  
            author = aut;  
            language = lang;  
        }  
}
```

```
class Book {  
    protected:  
        int bookID;  
        string title;  
        string publisher;  
        int publicationYear;  
        string ISBN;  
        string condition;  
        string status;  
}
```

Inheritance

- Student and NonStudent classes inherit from the User class, sharing attributes like userID, password, and other methods.

```
class NonStudent: public User {
private:
    string job;
    string field;
public:
    NonStudent(int id=0, string pass="", PersonalInfo pI=PersonalInfo(),
               job = j;
               field = f;
    }
```

```
class User {
protected:
    int userID;
    string password;
    PersonalInfo pI;
    Book **owned;
    Book **exchanged;
    Review *re;
    Wishlist *wish;
public:
    int booknum;
    int wishnum;
    int exnum;
```

```
class Student: public User {
private:
    stringedulvl;
    string eduIns;
public:
    Student(int id=-1, string pass="", PersonalInfo pI=PersonalInfo(),
            edulvl = lvl;
            eduIns = ins;
    }
```

Polymorphism

- In the Book class, the **dispBookDetails()** method is set as a virtual method.
- Both Dictionary and Encyclopedia classes have methods with the same name, parameters, and return type.
- These methods override the Book class method.

```
class Book {  
    virtual void dispBookDetails() const {  
        cout << setw(11) << setprecision(6) << bookID  
            << setw(20) << title  
            << setw(13) << publisher  
            << setw(8) << publicationYear  
            << setw(15) << ISBN  
            << setw(15) << condition  
            << setw(15) << status;  
    }  
};
```

```
class Encyclopedia: public Book {  
    void dispBookDetails() const override {  
        Book::dispBookDetails();  
        cout << setw(15) << author  
            << setw(15) << language  
            << endl;  
    }  
};
```

```
class Dictionary: public Book {  
    void dispBookDetails() const override {  
        Book::dispBookDetails();  
        cout << setw(15) << authorTeam  
            << setw(15) << pair_language  
            << endl;  
    }  
};
```

Thanks For Listening

